Using Android Data and Storage APIs

Tanusree Nath
PhD Research Scholar
Department of Computer Science, BHU

Google scholar: https://scholar.google.com/citations?user=h80-tYYAAAAJ&hl=en
LinkedIn: https://www.linkedin.com/in/tanusree-nath-5447291a7/

The Android Storage Landscape

Understanding Android storage is crucial for managing data security, user privacy, and app performance. The modern Android ecosystem differentiates between **where** data lives and **who** can access it.

Three types of storage designs:

1. Internal vs External
2. App-specific storage
3. Shared storage

Internal vs. External: The Modern Definition

In early Android, "External" meant a physical SD card. Today, "External" refers to **shared storage space** on the device's built-in flash memory that is accessible by the user via a PC or file manager.

Feature	Internal Storage	External Storage (App-Specific)
Location	/data/user/0/package_name/	/storage/emulated/0/Android/data/package_name/
Visibility	Hidden from user and other apps.	Visible to user (via File Manager/PC).
Permissions	None required.	None required (since Android 4.4).
Uninstall	Data is deleted.	Data is deleted.
Best Use	Sensitive data, private databases.	Large files not needed after uninstall.

App-Specific Storage (Private)

- This is the "walled garden" for your application. No other app can peek inside these folders.
- **Internal Storage:** This is the most secure location. Use this for SharedPreferences, internal databases (SQLite/Room), and any sensitive user information.
  - *Key Command:* context.getFilesDir() or context.getCacheDir().
- **External App-Specific Storage:** Located in the Android/data/ directory. While technically "external," other apps cannot access these files on modern Android versions (API 30+). It is perfect for large files that the user doesn't need to see in their gallery, like game assets or temporary video buffers.
  - *Key Command:* context.getExternalFilesDir().

Shared Storage (Public)

- If you want a photo to appear in the user’s Gallery or a PDF to be found in their Downloads folder, you must use **Shared Storage**.
- **Important:** On modern Android (Scoped Storage), you cannot use direct file paths (e.g., /storage/emulated/0/Pictures/my_file.jpg) to write to these areas.

How to Access Shared Storage:

- **MediaStore API:** Use this for well-defined media types like **Images, Video, and Audio**. It treats the storage like a database where you "insert" a record of your file.
- **Storage Access Framework (SAF):** Use this for everything else (PDFs, ZIPs, custom doc types). It invokes the system file picker, letting the **user** choose exactly where the file should be saved or opened. This grants your app temporary "URI" access rather than broad folder permissions.

Types of data to store

1. Simple App Settings (Key-Value) - small bits of data like "Is Dark Mode on?" or "User’s font size.“
   - **Tool: Jetpack DataStore**
2. . Complex & Searchable Data (Structured) - This is for data that looks like a spreadsheet: rows and columns, like a list of students, a product catalog, or a chat history.
   - **Tool: Room Persistence Library**
3. Private Large Files - Think of video game assets, temporary PDF reports, or downloaded files that only *your* app should use.
   - **Tool: App-Specific Internal/External Storage**
4. Public Media (Photos, Video, Audio) - If you want a photo to show up in the user's Gallery app or a song to be found by a music player.
   - **Tool: MediaStore API**
5. User Documents (PDFs, ZIPs, Picked Files)
   - **Tool: Storage Access Framework (SAF)**

Jetpack DataStore: The Modern Key-Value Solution

As of Android's modern development standards, **SharedPreferences is deprecated** in favor of Jetpack DataStore. DataStore addresses the primary flaws of its predecessor, namely, the lack of a reactive API and the risk of runtime exceptions caused by synchronous disk I/O on the main thread.

```
Jetpack DataStore (library)
    │
    ├── Preferences DataStore (key-value, no schema)
    └── Proto DataStore (typed objects, requires schema)
```

Room Persistence Library – for structured data

For apps requiring structured data with complex relationships, the **Room Persistence Library** is the industry standard.

Room is a **tool for Android apps** that helps you:

- Store data in a database
- Retrieve it easily
- Avoid complex coding

It works on top of **SQLite** (the actual database system)

Using SQLite directly is **Complicated**, **Error-prone** and **Requires a lot of repetitive code.**

Room makes it **Easier, Safer** and **Cleaner**

Room Persistence Library – The Three Pillars of Room

- **Entity:**
  - Defines what your data looks like
  - Like a **table in Excel**
  - Each object = one row
  - Each field = one column

```
@Entity
data class Student(
    @PrimaryKey val id: Int,
    val name: String
)
```

- **DAO (Data Access Object):** Defines **what operations you can perform**

  Think of DAO as A **menu of actions** (insert, read, update, delete)

```
@Dao
interface StudentDao {

    @Insert
    fun insert(student: Student)

    @Query("SELECT * FROM student")
    fun getAll(): List<Student>
}
```

Room Persistence Library – The Three Pillars of Room

- **Database:** The main access point which Connects everything together.

```
@Database(entities = [Student::class], version = 1)
abstract class AppDatabase : RoomDatabase() {
    abstract fun studentDao(): StudentDao
}
```

How it works (step-by-step)

- Define data (Entity)
- Define operations (DAO)
- Create database
- Use DAO to store/retrieve data

MediaStore API: The Global Gallery

If your app's purpose is to "contribute" to the device like a camera app saving a photo or a voice recorder saving a clip, you use the **MediaStore API**.

- **No Permissions for Contributions:** Your app can *write* media files to the shared Pictures, Video, or Audio folders without any user prompts.
- **The Media Collection:** It acts like a database. You don't "save a file"; you "insert a row" into the MediaStore.Images or MediaStore.Video collection.
- **Attribution:** Android remembers your app created that file. You can modify or delete files your app owns without extra permissions.

Storage Access Framework (SAF): The Document Picker

- For files that don't fit into "Media" (like PDFs, ZIPs, or custom text files), we use the **Document Picker**.
- Apps don’t access files directly.
  They ask the system, and the user grants access.

How SAF works (simple flow)

- App sends a request (Intent)
- System shows file picker UI
- User selects a file
- App gets access via a **URI**

Summary: Choosing the Right Storage API

"Cheat Sheet" for architectural decisions. Selecting the correct API ensures that your app is performant, follows modern Android security standards, and provides a seamless user experience.

Data Type	Recommended API	Why?
Simple User Settings	**Jetpack DataStore (Preferences)**	Replaces SharedPreferences. It is asynchronous, thread-safe, and avoids UI jank.
Complex Objects	**Jetpack DataStore (Proto)**	Offers full type safety via Protocol Buffers. Prevents runtime crashes from data type mismatches.
Large/Searchable Data	**Room (SQLite)**	Provides the full power of SQL. Essential for efficient indexing, complex relationships, and massive datasets.
User Photos/Videos	**MediaStore API**	The "Public Gallery" API. Allows your media to be indexed and seen by other apps (like Google Photos).
Documents / PDFs	**Storage Access Framework (SAF)**	Prioritizes user privacy. Let's the user pick specific files without giving your app broad storage access.

Security and Best Practices - Cleanup: Managing the cacheDir

- **Internal Cache:** Files saved in context.cacheDir are private but **volatile**.
- **System Awareness:** Explain that if a user's phone runs out of space, Android will automatically delete files in the cache folders of various apps to reclaim storage.
- **Best Practice:** Use this for downloaded thumbnails, temporary zip files, or any data that can be re-fetched if deleted.

Security and Best Practices - Storage Transparency (Android 15+)

Modern Android empowers users to see exactly *what* is taking up space. You can integrate this into your app’s "Settings" or "About" page.

StorageStats API: Use StorageStatsManager.getAppBytesByDataType() to break down your app's footprint into:

- **APK/Code:** The size of the app itself.
- **Data:** User-generated content and databases.
- **Cache:** Temporary files.

Security and Best Practices - The "No-Permission" Mindset

The most important lesson for 2026 is that **permissions are a friction point.**

Golden Rule: If a system picker (like SAF) can do the job, **do not** ask for a permission.

- **Legacy Permissions:** READ_EXTERNAL_STORAGE is essentially dead for apps targeting modern APIs. **The Benefit:**
- By using Intent.ACTION_OPEN_DOCUMENT (SAF) or the PhotoPicker, your app:
  - Requires **zero** manifest permissions.
  - Builds immediate **user trust**.
  - Is **future-proofed** against further Android OS privacy restrictions.

Thank You